

ESTEFANIA ROMERO L.

IDBR SI

TRABAJO 10

Introducción a Swagger



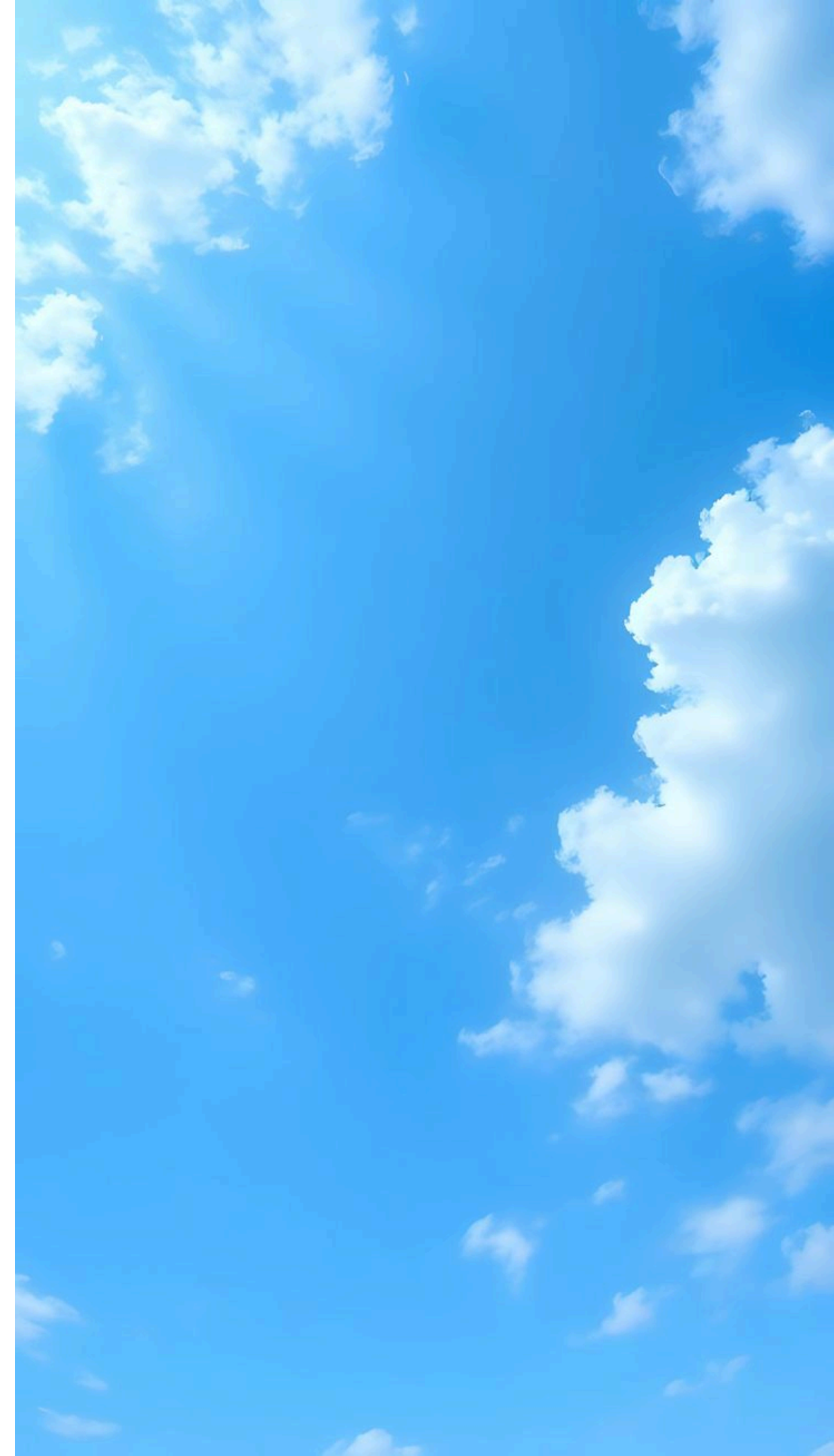
FLASK

Es un microframework ligero y de código abierto escrito en python, diseñado para crear aplicaciones web, APIs y microservicios de manera rápida y sencilla. Se considera "micro" porque no impone estructuras estrictas ni dependencias obligatorias, ofreciendo flexibilidad para añadir componentes externos según sea necesario.

¿Que es un template en flask?

Un template (plantilla) en Flask es un archivo HTML que contiene estructura estática junto con marcadores de posición (placeholders) para datos dinámicos.

son archivos que contienen datos estáticos así como marcadores de posición para datos dinámicos. Una plantilla se renderiza con datos específicos.



¿Como hacer una aplicacion en flask?

1-Crea una carpeta para tu proyecto y entra en ella.

2- Crea un entorno virtual
(recomendado): `python -m venv venv`

3- Activa el entorno:

Windows: `venv\Scripts\activate`

macOS/Linux: `source`

`venv/bin/activate`

4- Instala el Flask: `pip install flask`





Request

Es una petición que normalmente se hace a un servidor o una API (una aplicación en Internet que sirve recursos), el request sale de nuestro navegador, con ciertos datos con el protocolo HTTP

Ejemplo: Queremos mostrar el nombre de los usuarios; entonces necesitamos esos datos. Para obtenerlos hacemos un 'request' (le pedimos al servidor que nos de esos datos) o ect.



Cuales son los tipos de request y su uso

GET: Solicita o recupera datos de un recurso específico. No debe alterar el estado del servidor (es un método *Safe*).
Uso: Cargar una página web, consultar información en una API.

POST: Envía datos al servidor para crear un nuevo recurso o procesar información.
Uso: Enviar un formulario de registro, realizar una compra.

PUT: Reemplaza todos los datos actuales de un recurso existente por los datos enviados en la solicitud.
Uso: Actualizar el perfil completo de usuario.

DELETE: Elimina un recurso específico en el servidor.
Uso: Borrar un post, eliminar un archivo.

GET: se utiliza para solicitar y recuperar datos de un servidor sin modificarlos. Funciona enviando una URL específica, a menudo con parámetros de consulta (query string) para filtrar la información

POST: es un método HTTP utilizado para enviar datos desde un cliente (navegador, app) a un servidor, con el objetivo de crear, actualizar o procesar información

Funcionamiento de GET, POST, PUT Y DELETE

PUT: se utiliza para **actualizar un recurso existente** o crear uno nuevo si no existe, reemplazándolo completamente con los datos proporcionados en el cuerpo de la solicitud.

DELETE: se utiliza para solicitar la eliminación permanente de un recurso específico (como un usuario, archivo o registro de base de datos) identificado por una URL en el servidor.

Swagger

es un conjunto de herramientas de código abierto basado en la especificación OpenAPI que ayuda a diseñadores, desarrolladores y evaluadores a definir, crear, documentar y consumir APIs RESTful de manera eficiente.

Permite generar documentación interactiva automáticamente, facilitando la visualización y prueba de los puntos finales (endpoints) de la API directamente desde el navegador.

Como crear una Rest API con swagger

Pasos generales para crear una API con Swagger:

- **Diseño (Enfoque Design-First):** Utiliza Swagger Editor para crear un archivo YAML o JSON siguiendo la especificación OpenAPI, definiendo rutas (paths), métodos (GET, POST), parámetros y respuestas.
- **Desarrollo (Enfoque Code-First - Ejemplo Node.js):**
 - a. Inicia el proyecto: `npm init -y`.
 - b. Instala Swagger: `npm install swagger-ui-express swagger-autogen`.
 - c. Configura el archivo `swagger.js` para auto-generar la documentación basada en tus rutas Express.
 - d. Añade el middleware en tu archivo principal (`index.js`): `app.use('/docs', swaggerUi.serve, swaggerUi.setup(swaggerFile))`.
- **Implementación en Spring Boot (Java):**
 - e. Añade la dependencia `springdoc-openapi-ui` al archivo `pom.xml`.
 - f. Swagger detectará automáticamente los controladores y generará la documentación al ejecutar la aplicación.

Como utilizar swagger con Flask

1. Instalar dependencias : pip install flask flasgger,
2. Configurar la aplicacion flask

Formas de Documentar Endpoints

- **Comentarios YAML (Docstrings):** Como se muestra arriba, usando `""" --- YAML """` directamente bajo la función.
- **Decoradores Flasgger:** Usando `@swag_from` para cargar la documentación desde un archivo YAML separado o diccionari



Como conectar flask con Mysql

- 1- Instalar dependencias: `pip install Flask flask-mysqldb`
- 2- Configurar la aplicacion Flask
- 3- Ejecutar consultas.